

Lab 3 — Architecture Selection & Component Modelling Report

NAME : SIRI KRISHNA C R

Srn: PES1UG24AM276

1. System Overview

The system being designed is a **Pharmacy Expiry & Re-order Dispatch Engine**. Its main purpose is to support pharmacy checkout, medicine batch management, expiry monitoring, stock-level monitoring, and automatic replenishment.

The system must enforce **First-Expired-First-Out (FEFO)** during medicine dispensing, continuously monitor SKU stock against configurable safety thresholds, automatically generate and dispatch purchase orders when stock falls below the threshold, identify batches approaching expiry, and update inventory when ordered stock is received.

The system therefore contains several distinct responsibilities:

- Customer/pharmacy-clerk interaction
- Medicine checkout and FEFO selection
- Inventory and batch management
- Stock-level monitoring
- Near-expiry monitoring
- Purchase-order generation

UML Component Diagram



2. Architectural Styles Considered

The Lab 3 handout identifies three architectural styles for consideration: **Layered Architecture**, **Microservices Architecture**, and **Client-Server Architecture**.

2.1 Layered Architecture

Layered architecture organizes the system into logical layers such as **Presentation**, **Business**, and **Data**.

Advantages for this system:

- Clear separation of responsibilities
- Easy to understand and maintain
- Business rules can be separated from user-interface and data-management concerns
- Appropriate for a traditional application with well-defined business functionality

Disadvantages:

- Communication between layers can introduce some performance overhead
- Layers can become dependent on one another

- Scaling individual functions independently is more difficult

These characteristics are also identified in the Lab 3 handout.

2.2 Microservices Architecture

Microservices divides the system into small, independently deployable services.

Advantages:

- Individual services can be scaled independently
- Fault isolation is possible
- Different technologies can be used for different services
- Services can be deployed independently

Disadvantages for this scenario:

- Greater operational complexity
- Network communication introduces latency
- Maintaining consistent data across services becomes more difficult

These trade-offs are specifically identified in the lab handout.

For this pharmacy system, introducing multiple independently deployed services would add complexity that is not necessary for the relatively focused scope of the application.

2.3 Client-Server Architecture

Client-server architecture uses clients that communicate with a centralized server.

Advantages:

- Centralized control
- Simple deployment
- Easier centralized data consistency

Disadvantages:

- The central server can become a single point of failure
- Scalability can become a bottleneck
- Fault tolerance is more limited

These characteristics are given in the Lab 3 handout.

3. Selected Architecture — Layered Architecture

I chose Layered Architecture for the Pharmacy Expiry & Re-order Dispatch Engine.

Layered Architecture is the most suitable choice because the pharmacy system has **clearly separated responsibilities** and contains several important business rules that should remain independent from the user interface and external supplier communication.

The architecture is divided into:

Presentation Layer



Business Layer



Data / Integration Layer



External Supplier

This organization follows the layered approach described in the Lab 3 handout.

4. Justification for Selecting Layered Architecture

Reason 1 — Clear separation of pharmacy responsibilities

The system performs several logically different operations.

For example, **FEFO selection** is a business rule, while displaying medicines to the Pharmacy Clerk is a presentation responsibility. Similarly, storing batch quantities and expiry dates is different from monitoring whether stock has fallen below a safety threshold.

Layered Architecture allows these responsibilities to be separated rather than placing all functionality into one large module.

For example:

Pharmacy Clerk Interface



Dispensing / Checkout Manager



Inventory & Batch Manager

The Pharmacy Clerk Interface handles interaction, the Checkout Manager handles checkout and FEFO logic, and the Inventory & Batch Manager handles inventory information.

This separation makes individual components easier to understand, modify and maintain.

Reason 2 — Suitable for the system's rule-based business logic

The pharmacy system contains important rules that must be applied consistently.

The most important example is **FEFO**. During checkout, the system must select the available batch with the nearest expiry date. If the first batch does not contain enough stock, the system must use the remaining quantity and then select the next earliest-expiring batch.

Similarly, the Stock Monitor must compare stock against a safety threshold, while the Purchase Order / Re-order Manager handles replenishment when the threshold is breached.

Keeping these responsibilities in separate business components makes the architecture easier to maintain and reduces the risk that changes to one business rule will affect unrelated parts of the system.

Reason 3 — Appropriate level of complexity

Microservices would provide independent scaling and fault isolation, but they would also introduce additional operational and network complexity. The lab specifically identifies **network latency, operational complexity, and data consistency challenges** as disadvantages of microservices.

For this system, the main requirement is to organize a well-defined pharmacy application rather than operate many independently deployed services.

Therefore, Layered Architecture provides a better balance between **simplicity, maintainability, and separation of concerns**.

5. Components Used

Pharmacy System Components

1. Pharmacy Clerk Interface (Presentation Layer)

- User interface for clerks
- Functions: search medicines, view batches, enter quantity, checkout, display results

2. Dispensing / Checkout Manager (Business Layer)

- Core sales processor
- Tasks: checkout, FEFO batch selection, handle multiple batches, reduce stock, record sales

3. Inventory & Batch Manager (Data/Integration Layer)

- Manages stock and batch info
- Tasks: maintain records, expiry dates, update/add batches, provide FEFO data

4. Stock Monitor (Business Layer)

- Monitors stock vs. thresholds

- Triggers replenishment when stock is low (FR-002)
- 5. Near-Expiry Monitor (Business Layer)
 - Detects batches nearing expiry
 - Configurable window (default 30 days), generates near-expiry report
- 6. Purchase Order / Re-order Manager (Business/Integration Layer)
 - Handles automatic replenishment
 - Tasks: receive requests, identify SKU, generate & dispatch purchase orders (FR-003)
- 7. Supplier Integration (Data/Integration Layer)
 - Communicates with external supplier
 - Tasks: send purchase orders, authenticate supplier, receive delivery info (TLS 1.2+)

External System:

- Inventory Supplier → Provides replenishment and confirms deliveries for new batches.

6. Interfaces Used

The component diagram uses seven interfaces.

Interface	Provided by	Required by	Purpose
CheckoutService	Dispensing / Checkout Manager	Pharmacy Clerk Interface	Processes medicine checkout
BatchInventoryService	Inventory & Batch Manager	Dispensing / Checkout Manager	Provides batch and inventory information
ExpiryMonitoringService	Inventory & Batch Manager	Near-Expiry Monitor	Provides expiry/batch information
StockMonitoringService	Inventory & Batch Manager	Stock Monitor	Provides stock information
ReorderService	Purchase Order / Re-order Manager	Stock Monitor	Initiates replenishment
SupplierOrderAPI	Supplier Integration	Purchase Order / Re-order Manager	Communicates purchase orders
DeliveryConfirmationService	Supplier Integration	Inventory & Batch Manager	Provides delivery information for inventory update

The UML notation distinguishes **provided interfaces using the ball notation** and **required interfaces using the socket notation**.

7. Security Advantage

A major security advantage of the layered architecture is that **supplier communication is isolated within the Supplier Integration component**.

Supplier-related communication can therefore be controlled in one place, where authentication and encrypted communication can be enforced.

This is particularly important because the system requirement specifies:

- TLS 1.2 or higher
- Supplier authentication
- Secure purchase-order dispatch

The requirement also specifies that purchase-order requests should be dispatched within two seconds after a threshold breach is detected.

Centralizing supplier communication in the Supplier Integration component makes it easier to enforce these security controls consistently.

8. Performance Advantage

The architecture separates **stock monitoring** from the Pharmacy Clerk Interface.

The Stock Monitor can independently check SKU stock levels and initiate the re-order process when a threshold is breached. This prevents the Pharmacy Clerk from having to manually check stock and initiate every replenishment.

The system requirement specifies that a purchase order should be generated and dispatched within **2 seconds of detecting a threshold breach**.

Therefore, separating:

Inventory & Batch Manager



Stock Monitor



Purchase Order / Re-order Manager

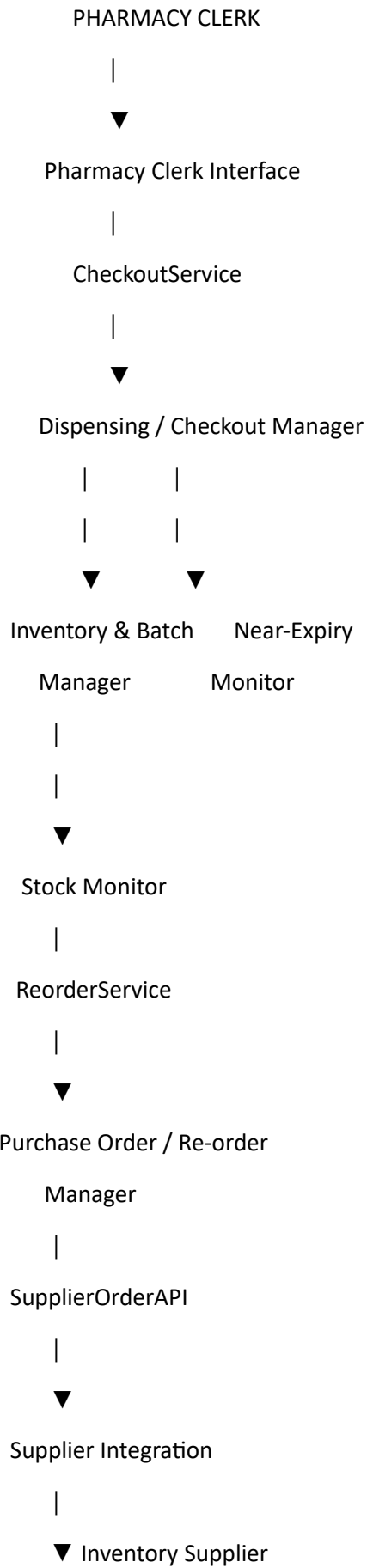


Supplier Integration

supports a fast automated replenishment workflow.

9. Overall Architecture Flow

The overall system interaction can be summarized as:



The checkout flow follows the use-case in which the clerk searches for medicine, the system identifies available batches, applies FEFO, updates stock, records the sale, and informs the Stock Monitor.

10. Conclusion

Layered Architecture was selected because it provides clear separation of presentation, business, inventory, and external-integration responsibilities while keeping the architecture understandable and maintainable.

The architecture is particularly appropriate because the system contains well-defined business rules such as **FEFO selection, stock-threshold monitoring, near-expiry detection, and automatic purchase-order generation**. Separating these responsibilities into dedicated components reduces coupling and makes future maintenance easier.

The architecture also provides a clear location for security controls through the Supplier Integration component and supports the required fast automated replenishment process by separating stock monitoring and purchase-order processing.